

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Bilete**

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Introducere și Contextul Sistemului	2
1.1	Harta de Context (Context Map)	2
2	Arhitectura Internă a Microserviciului Bilete	2
2.1	Diagrama de Pachete (Arhitectura Hexagonală)	2
2.2	Diagrama de Clase Detaliată	3
3	Design-ul Bazei de Date (Persistența)	4
4	Modelarea Domeniului (Domain-Driven Design)	4
4.1	Aggregate Root — Clasa Bilet	4
4.2	Value Objects	4
5	Șabloane de Proiectare (Design Patterns) Utilizate	5
5.1	Șablonul Adaptor (Adapter / Wrapper)	5
5.2	Șablonul Singleton (Gestionat prin IoC)	5
6	Concluzii	5

1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale, fiecare acționând ca un microserviciu independent: *Zboruri*, *Bilete*, *Utilizatori*, *Notificări*, *Statistici* și *Export*.

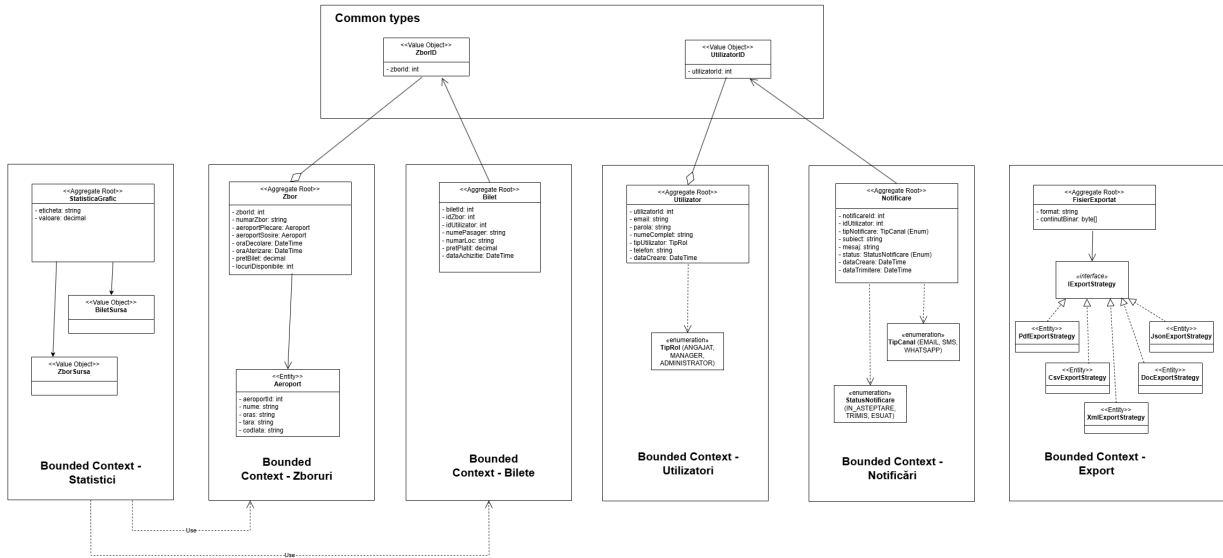


Figura 1: Harta de context a sistemului global de gestiune

Justificarea deciziei: Această decizie arhitecturală previne crearea unui monolit rigid. Contextul **Bilete** (vizat exclusiv în acest document) se axează strict pe gestionarea achizițiilor și alocarea locurilor pentru pasageri. El nu are responsabilitatea de a stoca detalii despre rutele avioanelor sau datele personale sensibile ale utilizatorilor. În schimb, folosește tipuri comune (*Common types*) precum `ZborID` și `UtilizatorID` pentru a realiza asocierile. Microserviciul de bilete comunică cu cel de zboruri doar pentru a verifica/rezerva locuri, demonstrând o decuplare eficientă.

2 Arhitectura Internă a Microserviciului Bilete

2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Bilete** respectă cu strictețe principiul Inversiunii Dependențelor (*Dependency Inversion Principle*), organizându-și straturile astfel încât regulile de business să fie complet izolate de infrastructura tehnică.

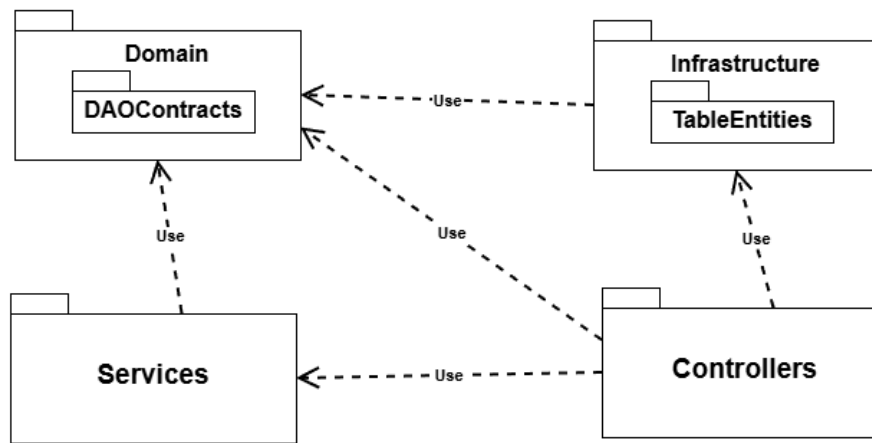


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

Justificarea deciziei: Săgețile de tip *-Use-* din figura 2 indică direcția dependențelor. Pachetul central **Domain** conține contractele (interfețele din **DAOContracts**) și modelele pure, neavând nicio dependență spre exterior. Pachetul **Infrastructure** comunică cu baza de date prin **TableEntities** și depinde de **Domain** pentru a implementa logica de salvare. Această decuplare permite schimbarea bazei de date sau a clientului HTTP extern fără a atinge codul din **Services** sau **Domain**.

2.2 Diagrama de Clase Detaliată

Pentru a expune în mod granular structura internă, am mapat atributele, metodele și asocierile din fiecare strat al microserviciului Bilete.

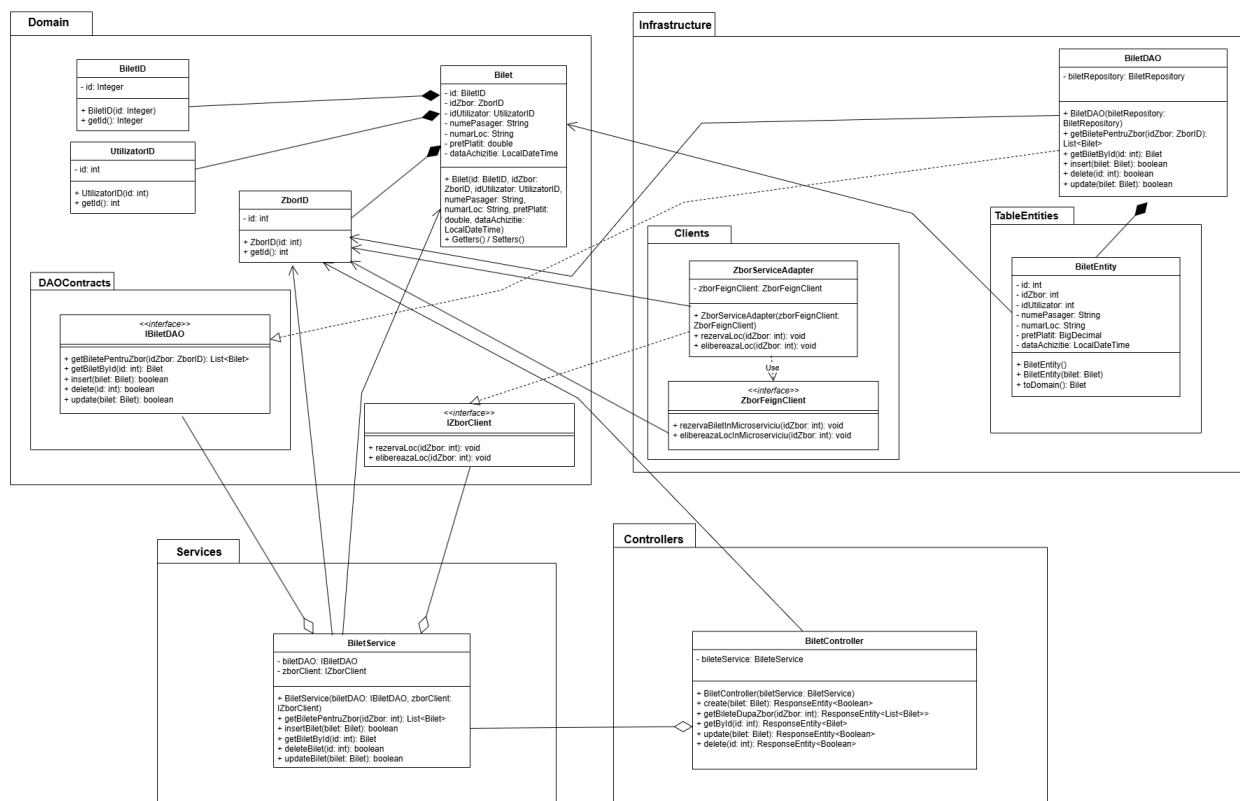


Figura 3: Diagrama de clase detaliată a microserviciului Bilete

Justificarea deciziei: Figura 3 prezintă fluxul complet al datelor. **BiletController** preia cererile și le delegă către **BiletService**. La rândul său, serviciul orchestrează entitatea centrală **Bilet** (care înglobează

Value Objects precum `BiletID`, `ZborID`, `UtilizatorID`) și apelează contractele din `IBiletDAO` și `IZborClient`. Implementările concrete, precum `BiletDAO` și `ZborServiceAdapter`, rezidă în infrastructură, ascunzând detaliile framework-urilor (JPA, FeignClient).

3 Design-ul Bazei de Date (Persistența)

Stocarea biletelor este gestionată printr-un tabel relațional simplu și eficient, proiectat pentru a respecta natura distribuită a microserviciilor.

bilet	
PK	<u>id: int</u>
	id_zbor: int
	id_pasager: int
	nume_pasager: varchar(100)
	numar_loc: varchar(10)
	pret_platit: decimal(10,2)
	data_achizitie: datetime

Figura 4: Diagrama Entitate-Relație (ERD) pentru tabelul bilet

Justificarea deciziei: Tabela `bilet` folosește o cheie primară auto-incrementată (`id`). Cel mai important aspect arhitectural este stocarea referințelor `id_zbor` și `id_pasager` ca simple coloane întregi, fără constrângeri stricte de *Foreign Key* la nivel de bază de date fizică. Deoarece zborurile și utilizatorii trăiesc în baze de date separate aparținând altor microservicii, legătura este una logică (*soft link*), garantată prin validările din aplicație (Domain), asigurând astfel independența totală a bazei de date aferente biletelor. De asemenea, date specifice tranzacției, precum `numar_loc` și `pret_platit`, sunt reținute local pentru un acces rapid și generarea istoricului de achiziții.

4 Modelarea Domeniului (Domain-Driven Design)

Nucleul de business al aplicației se bazează pe reguli stricte definite în **Domain**.

4.1 Aggregate Root — Clasa Bilet

Clasa `Bilet` funcționează ca un *Aggregate Root* imutabil. Prin intermediul constructorului său, entitatea garantează următoarele reguli de afaceri (*Guard Clauses*):

- **Validarea datelor pasagerului:** Numele pasagerului și numărul locului sunt câmpuri obligatorii (nu pot fi nule sau goale).
- **Validarea prețului:** Prețul plătit nu poate fi o valoare negativă.
- **Asocierea obligatorie:** Un bilet nu poate exista independent; trebuie neapărat să aibă un `ZborID` și un `UtilizatorID` valide asociate în momentul instanțierii.

4.2 Value Objects

Pentru a elimina riscul erorilor de tip *Primitive Obsession* (ex: trimiterea din greșeală a ID-ului de zbor în locul celui de bilet), am modelat `BiletID`, `ZborID` și `UtilizatorID` ca **Value Objects**. Ele încapsulează câte o valoare primitivă (`int`) și o protejează la nivel de compilator.

5 Șabloane de Proiectare (Design Patterns) Utilizate

5.1 Șablonul Adaptor (Adapter / Wrapper)

Acest pattern este esențial în microserviciul nostru pentru a respecta arhitectura curată, decuplând nucleul de business de comunicarea HTTP și baza de date.

Explicație și Justificare: În figura 3, interfața `IZborClient` reprezintă un port în interiorul domeniului, definind cerința de a rezerva sau elibera un loc. Clasa `ZborServiceAdapter` acționează ca adaptorul concret din infrastructură; ea implementează interfața pură, dar în spate se folosește de tehnologia `ZborFeignClient` pentru a trimite efectiv cereri REST către microserviciul Zboruri. Dacă pe viitor dorim să renunțăm la OpenFeign și să folosim RabbitMQ sau gRPC, vom modifica doar clasa `ZborServiceAdapter`, lăsând `BiletService` intact. Același principiu se aplică și la `BiletDAO`, care ascunde complexitatea `BiletRepository` (Spring Data JPA) față de serviciu.

5.2 Șablonul Singleton (Gestionat prin IoC)

Ciclul de viață al componentelor centrale este gestionat sub formă de *Singleton* prin intermediul containerului IoC (Inversion of Control) din Spring.

Explicație și Justificare: Clasele marcate cu adnotări precum `@RestController` (`BiletController`), `@Service` (`BiletService`) și `@Component` (`BiletDAO`, `ZborServiceAdapter`) sunt instanțiate o singură dată la pornirea aplicației. Această decizie optimizează masiv consumul de memorie. Mai mult, deoarece aceste clase sunt *stateless* (nu păstrează date de stare interne, ci doar rulează comenzi și efectuează operațiuni tranzacționale pe baza parametrilor primiți), abordarea Singleton este complet sigură într-un mediu concurent (multithreading), capabil să proceseze sute de rezervări simultan.

6 Concluzii

Arhitectura microserviciului **Bilete** reprezintă un exemplu robust de *Clean Architecture*. Prin separarea clară pe pachete, utilizarea intensivă a pattern-ului Adaptor pentru interacțiunea cu alte microservicii și baze de date, precum și aplicarea principiilor de validare a agregatelor din Domain-Driven Design, am obținut un sistem scalabil, sigur la tipizare și independent tehnic. Această abordare permite extinderea facilă a funcționalităților agenției fără a risca stabilitatea modulelor deja implementate.